

The Math Behind the Testbench

Mapping Physics to DSP48A1 Verification

Omar Sadek

1 Introduction

This document outlines the kinematic equations used to verify the Multiply-Accumulate (MAC) functionality of the DSP48A1 slice. By simulating a dual-projectile tracking system, we demonstrate how discrete-time calculus (Euler integration) is translated into a Verilog testbench to evaluate the hardware's numerical processing capabilities.

2 Mathematical Foundation: 2D Kinematics

In classical mechanics, 2D projectile motion is analyzed by separating the vertical and horizontal components. The simulation calculates the position (X, Y) iteratively over a discrete time step Δt .

Vertical Motion (Y-axis)

The vertical motion is subjected to gravitational acceleration g . Therefore, the vertical velocity V_y decreases iteratively:

$$V_y[n] = V_y[n - 1] - (g \times \Delta t) \quad (1)$$

The altitude Y is accumulated iteratively based on the updated velocity:

$$Y[n] = Y[n - 1] + (V_y[n] \times \Delta t) \quad (2)$$

Horizontal Motion (X-axis)

Assuming no air resistance, there is no horizontal acceleration. Thus, the horizontal velocity V_x remains constant:

$$V_x[n] = V_x[0] = \text{constant} \quad (3)$$

The horizontal distance X is updated through simple linear accumulation:

$$X[n] = X[n - 1] + (V_x \times \Delta t) \quad (4)$$

3 Mapping Mathematics to the Testbench

In this simulation, the testbench computes the horizontal distance X and updates the vertical velocity V_y via software-like behavior. However, the heavy computational lifting for the vertical accumulation (Equation 2) is offloaded to the DSP48A1 hardware.

The standard MAC operation executed by the DSP48A1 is defined as:

$$P_{new} = P_{old} + (A \times B) \quad (5)$$

Table 1 illustrates the direct translation from the physical variables to the Verilog signals.

Physical Variable	Verilog Signal	Testbench / Hardware Mapping
$V_y[n]$	A_vya / A_vyb	Driven by the testbench loop, feeds into DSP Port A.
Δt	B_dt	Constant time step (18' sd1), feeds into DSP Port B.
$Y[n]$	P_ya / P_yb	Output from DSP Port P, represents the accumulated altitude.
g	gravity	Used to decrement velocity per clock cycle.

Table 1: Mapping of Kinematic Equations to Signals

Fixed-Point Scaling Strategy

Hardware DSP blocks operate on integers and do not support floating-point arithmetic. To accurately represent fractional physical values like standard gravity ($g = 9.8 \text{ m/s}^2$), a simple fixed-point scaling approach is used.

All physical parameters are multiplied by a scale factor of 10 in the testbench. For example, gravity is defined as $g_{scaled} = 98$, and an initial velocity of 250 becomes 2500. After the hardware computes the scaled values, the Python script divides the extracted coordinates by 10.0 to restore the original decimal values for accurate plotting.

4 Case Study: Analytical vs. Hardware Simulation

To validate the DSP48A1 hardware output, an analytical calculation of the intersection point is performed. Before diving into the mathematics, it is important to outline the data extraction pipeline used in this verification environment.

Simulation Data Pipeline

During the simulation, the testbench calculates the (X, Y) coordinates for both projectiles at each clock cycle and saves them directly to a text file (`collision_data.txt`). When the projectiles intersect, the simulation stops and logs the exact intersection point with a `COLLISION` flag

As shown in Figure 1, the hardware outputs are successfully extracted as fixed-point integers (e.g., 50000 and 19420). A Python script is then utilized to parse this text

file, reversing the fixed-point scaling by dividing the integers by 10.0 to restore their floating-point precision for visualization.

collision_data.txt

+

FileEditView

0	0	80000	0
2500	1902	78500	1902
5000	3706	77000	3706
7500	5412	75500	5412
10000	7020	74000	7020
12500	8530	72500	8530
15000	9942	71000	9942
17500	11256	69500	11256
20000	12472	68000	12472
22500	13590	66500	13590
25000	14610	65000	14610
27500	15532	63500	15532
30000	16356	62000	16356
32500	17082	60500	17082
35000	17710	59000	17710
37500	18240	57500	18240
40000	18672	56000	18672
42500	19006	54500	19006
45000	19242	53000	19242
47500	19380	51500	19380
50000	19420	50000	19420
COLLISION	50000	19420	

Figure 1: Snippet of the `collision_data.txt` file showing the scaled outputs generated by the DSP48A1 testbench.

The testbench was initialized with the following unscaled physical parameters:

- **Interceptor (A):** $X_{a0} = 0$ m, $V_{xa} = 250$ m/s, $V_{ya0} = 200$ m/s.
- **Target (B):** $X_{b0} = 8000$ m, $V_{xb} = -150$ m/s, $V_{yb0} = 200$ m/s.
- **Environment:** $g = 9.8$ m/s², $\Delta t = 1$ s.

Calculating the Intersection Time (t)

A collision occurs when both horizontal and vertical coordinates match. By setting the horizontal position equations equal ($X_a = X_b$):

$$X_{a0} + (V_{xa} \times t) = X_{b0} + (V_{xb} \times t) \quad (6)$$

$$0 + 250t = 8000 - 150t \implies 400t = 8000 \implies t = 20 \text{ seconds} \quad (7)$$

Calculating the Intersection Coordinates

At $t = 20$ s, the exact horizontal collision point is:

$$X_{collision} = 250 \times 20 = 5000 \text{ meters} \quad (8)$$

For discrete-time Euler integration, the accumulated altitude at step n follows the arithmetic progression sum: $Y[n] = n \cdot V_{y0} - g \frac{n(n+1)}{2}$. Substituting $n = 20$:

$$Y_{collision} = (20 \times 200) - 9.8 \times \left(\frac{20 \times 21}{2} \right) = 4000 - 2058 = 1942 \text{ meters} \quad (9)$$

As illustrated in Figure 2, the Python script plotting the exact outputs from the DSP48A1 Verilog simulation successfully confirms the collision at $(X : 5000.0, Y : 1942.0)$, proving the mathematical precision of the hardware datapath.

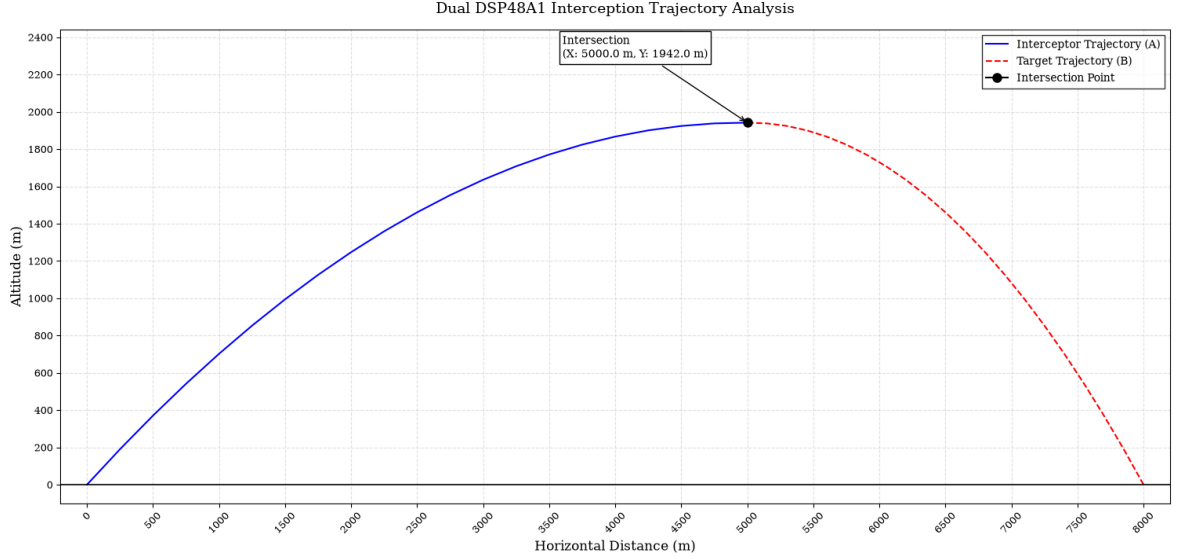


Figure 2: visualization of the DSP48A1 outputs showing exact collision coordinates.